



DARA: An Autonomous Multi-Agent Pipeline for Root-Cause Analysis and Verified Bug Resolution — Technical Report

Saranraj U · Chettinad College of Engineering, Chennai, India ·
saran17102005@gmail.com · github.com/saranraj1/DARA-v2

Draft v0.2 — July 2026



Architecture, configuration values, and design details in this draft are grounded in the actual DARA repository (README + project structure). Remaining **TODO** blocks mark places that need measured experimental results — do not share externally until those are filled with real numbers.

Abstract

Resolving software defects in production systems requires more than patch generation: faults must be detected from runtime signals, localized across service boundaries, fixed without introducing regressions or vulnerabilities, and shipped through a reviewable process. We present **DARA** (Distributed Architecture Root-cause Agent), an autonomous multi-agent pipeline that ingests errors from OpenTelemetry traces, GitHub webhooks, Slack alerts, or a REST API, and carries them through a deterministic 12-stage pipeline: root-cause analysis, patch synthesis, sandboxed validation with iterative self-healing, security auditing, and GitHub pull-request creation. The agent layer separates concerns across a DebuggerAgent (root-cause analysis with confidence scores), a FixerAgent (unified-diff synthesis with security-

constrained rewrites), a ReviewerAgent (quality review plus Bandit/Semgrep security audit), and a strategy subsystem that selects, proposes, and monitors fix strategies backed by a pattern library of learned fix templates. Safety is enforced by explicit gates: fixes are auto-approved only when reviewer recommendation, confidence (≥ 0.88), regression risk, and sandbox validation all align, while low-confidence diagnoses (< 0.35), critical blast-radius findings (computed by Neo4j graph traversal over the service topology), or repeated security failures escalate to a human via Slack and a human-in-the-loop review queue. DARA supports polyglot codebases (Python, Go, TypeScript via Tree-sitter), and ships with full observability (Prometheus, Grafana, Jaeger) and an RLHF-style feedback loop that records PR outcomes for continuous strategy improvement. We describe the architecture and safety design, analyze principal failure modes — most notably uncalibrated confidence scores and outcome-only semantic verification — and present an evaluation plan against SWE-bench Lite and recorded production traces.

1. Introduction

Most LLM-based program repair systems begin from a written issue report and end at a candidate patch. Production reality is wider on both ends: defects announce themselves as runtime signals (traces, alerts, webhook events) before anyone writes an issue, and a patch is only useful once it has been validated, security-audited, impact-assessed, and turned into a reviewable pull request. DARA is designed around this wider loop.

Three design principles shape the system:

1. **Separation of concerns across agents.** Diagnosis, synthesis, review, and strategy selection have different context requirements and failure modes; DARA assigns each to a dedicated agent with typed outputs (e.g., `RootCauseResult`, `ReviewResult`), which makes stages independently inspectable and replaceable.
2. **Autonomy behind explicit safety gates.** Full autonomy is permitted only in the region where the system is demonstrably reliable: the auto-approve gate requires a reviewer recommendation of `approve`, fix confidence ≥ 0.88 , low regression risk, and passing sandbox validation — all simultaneously. Everything else routes to escalation or human review.
3. **Learning from operation.** Every PR outcome (accepted / rejected / modified) is captured through a feedback API; successful fixes can be

promoted into a pattern library that provides a fast path for recurring error classes, and strategy performance is A/B-monitored over time.

Contributions.

1. An end-to-end architecture connecting multi-source error ingestion (OpenTelemetry, GitHub, Slack, REST) to verified pull-request creation via a deterministic 12-stage pipeline (Section 3).
2. A layered verification design combining Docker sandbox execution with iterative self-healing (≤ 3 iterations), static analysis (Ruff), and a security audit loop (Bandit + Semgrep with ≤ 2 security-constrained rewrites) (Section 5).
3. A graph-based blast-radius analysis over service topology (Neo4j) that gates merges by estimated cross-service impact (Section 5.3).
4. A confidence-gated autonomy model with defined escalation triggers, plus an operational analysis of its failure modes (Sections 5.4, 7).

2. Related Work



TODO: Read and cite before sharing. Position DARA against: (1) repository-level repair agents — SWE-agent (Yang et al., 2024), AutoCodeRover (Zhang et al., 2024), Agentless (Xia et al., 2024); (2) benchmarks — SWE-bench / SWE-bench Lite (Jimenez et al., 2024); (3) AIOps / automated root-cause analysis literature (trace-based RCA), which distinguishes DARA's runtime-signal entry point from issue-text-only systems; (4) LLM self-verification and calibration work relevant to Section 7.

DARA differs from repository-level repair agents along two axes. First, its entry point is a *runtime error signal* rather than a curated issue description — ingestion includes classification, normalization, and deduplication of errors arriving from live telemetry. Second, its exit criterion is stricter than “plausible patch”: a fix must survive sandbox tests, static analysis, a security audit, and an impact assessment before a PR is opened.

3. Pipeline Overview

The Orchestrator executes a deterministic 12-stage pipeline for every ingested error:

1. Load error from PostgreSQL
2. Pattern-library check — fast path if the matched template's historical success rate $\geq 85\%$
3. Build context bundle (semantic search + file contents)
4. DebuggerAgent → `RootCauseResult` with confidence score
5. Escalation gate — confidence < 0.35 → Slack escalation
6. FixerAgent → unified-diff patches; **6b.** blast-radius analysis (Neo4j graph traversal)
7. Sandbox validation (Docker) with iterative self-healing (≤ 3 loops); **7b.** static analysis (Ruff)
8. ReviewerAgent + security audit (Bandit + Semgrep); HIGH-severity findings trigger security-constrained FixerAgent retries (≤ 2)
9. Persist fix to PostgreSQL
10. Auto-approve gate (confidence + risk + sandbox all pass)
11. Slack notification
12. GitHub PR creation

Upstream of the pipeline, the **ingestion layer** (classifier, normalizer, deduplicator) converts heterogeneous error sources — OpenTelemetry traces, GitHub webhook events, Slack alerts, direct REST submissions — into a normalized error record with severity and type classification, deduplicating repeats before they consume pipeline resources.

4. Agent Layer

Agent	Responsibility
Orchestrator	Coordinates all 12 stages; owns escalation gates and auto-approve logic
DebuggerAgent	Analyzes error context; produces <code>RootCauseResult</code> with a confidence score

Agent	Responsibility
FixerAgent	Generates unified-diff patches; supports security-constrained rewrites
ReviewerAgent	Code-quality review + security audit; outputs <code>ReviewResult</code>
StrategyEvaluator	Selects fix strategy from historical success rates
StrategyGenerator	Proposes novel strategies via LLM when no known pattern matches
StrategyMonitor	Tracks live strategy performance for A/B evaluation
PatternMemory	Caches successful fix templates; fast-path resolution for recurring errors, with RLHF-driven memory consolidation

Context and retrieval. The context builder assembles a bundle for each agent from semantic search over a Qdrant vector index (sentence-transformer embeddings; CPU by default, GPU optional) plus raw file contents, capped at 8,000 context tokens (tracked with tiktoken). Polyglot support comes from Tree-sitter AST parsing for Python, Go, and TypeScript.

LLM backends. DARA is model-agnostic with automatic fallback: hosted inference via Groq, local inference via Ollama (default model `deepseek-coder-v2:16b`), Google AI Studio as fallback, and HuggingFace as an embeddings fallback. Per-stage retries are capped (`MAX_PIPELINE_RETRIES = 3`).

5. Verification and Safety Design

5.1 Sandbox Validation with Self-Healing

Every candidate patch is applied and tested in an ephemeral Docker sandbox (hard timeout 120 s). On failure, the failure evidence is fed back to the FixerAgent for refinement — at most 3 self-healing iterations before the pipeline abandons autonomous repair.

5.2 Static and Security Analysis

Patches that pass the sandbox undergo Ruff static analysis (S/E9 rule classes), then a security audit running Bandit and Semgrep. HIGH-severity findings do not simply fail the pipeline: they trigger up to 2 security-constrained rewrites, where the security findings are injected into the FixerAgent's prompt as hard constraints. If findings persist, the fix is marked `security_blocked` and escalated.

5.3 Blast-Radius Analysis

Before validation, a Neo4j-based analyzer traverses the service dependency graph (built from OpenTelemetry topology data) to estimate the cross-service impact surface of the proposed change, with fault-propagation modeling. A

`critical` blast radius forces human review regardless of confidence.

5.4 Autonomy Gates

Auto-approve (all conditions required): ReviewerAgent recommends `approve` $\wedge$ fix confidence $\geq 0.88 \wedge$ regression risk `low` $\wedge$ sandbox validation passed.

Escalation triggers: root-cause confidence < 0.35 ; strategy resolves to `human_escalation`; blast radius `critical`; security blocked after 2 retries.

Escalations flow to Slack and to the human-in-the-loop patch queue in the React admin console, where reviewers approve or reject diffs side-by-side; every decision is recorded in an immutable audit trail.

6. Platform and Implementation

- **Services:** Python 3.11 / FastAPI application layer with JWT authentication, role-based access control, and multi-tenancy; Celery background workers; Alembic-managed PostgreSQL schema.
- **Storage:** PostgreSQL (TimescaleDB) for errors/fixes, Redis for state/queues/caching, Qdrant for vector search, Neo4j for service topology, MinIO for object storage, Elasticsearch for logs.
- **Observability:** Prometheus metrics (`dara_pipelines_total` , `dara_active_pipelines` , `dara_pipeline_duration_seconds` , `dara_fixes_generated_total` , `dara_fixes_reviewed_total`), Grafana dashboards, OpenTelemetry instrumentation on all pipeline stages with Jaeger tracing, and structured JSON logging (structlog) shipped to Elasticsearch/Kibana.
- **Admin console:** a React-based console (`artisan-admin`) providing the RCA pipeline dashboard (KPIs, win rates, throughput), live ingestion log with stack traces and commit tags, the HITL patch queue, an interactive OTel topology call graph, and the audit trail.
- **Security posture:** JWT on all endpoints, Docker isolation for all validation, a service allowlist for GitHub repo resolution (path-traversal prevention), and optional HashiCorp Vault integration for secrets.

- **CI:** GitHub Actions workflows for linting (Ruff) and tests (pytest); end-to-end smoke tests and a benchmark harness (`run_benchmarks.py`) run against live infrastructure.

7. Failure-Mode Analysis

Operating DARA surfaced four systematic weaknesses, which we consider the system's research-relevant findings:

1. **Uncalibrated confidence.** The gates in Section 5.4 hinge on LLM-produced confidence scores (thresholds 0.88 and 0.35), but these scores are self-reported by the model, not calibrated estimates. A statistically grounded confidence — e.g., conformal prediction over historical fix outcomes — would give the auto-approve/escalate/abstain decisions actual guarantees. An initial 18-outcome calibration sample (Section 8.1) illustrates the concern: observed success did not increase monotonically with confidence (100% success in the 0.70–0.88 bucket vs. 40% at ≥ 0.88), and although the high-bucket failures were confounded by sandbox environment faults, that confound itself shows self-reported confidence is blind to environmental failure modes.
2. **Outcome-only semantic verification.** The sandbox validates against existing tests; Bandit/Semgrep validate against known vulnerability patterns. Neither establishes that a patch is semantically correct for the *intended* behavior — a patch that overfits weak test suites passes all gates ("correct outcome, wrong reason").
3. **Retrieval degrades out-of-domain.** Context quality falls on unfamiliar codebases, suggesting the flat vector index overfits repository-specific artifacts; the service topology graph (already in Neo4j) is not yet exploited for retrieval.
4. **Pattern-library staleness risk.** Fast-path templates promoted on historical success can silently rot as codebases evolve; the $\geq 85\%$ success-rate criterion is a point-in-time measurement without drift detection.



TODO: Attach 2–3 concrete, anonymized examples per failure mode from your run logs (a sandbox-passing but wrong patch; a low-confidence-but-correct diagnosis; a retrieval miss on an unfamiliar repo). These examples are what professors will probe.

8. Evaluation

8.1 Preliminary Measurements (July 2026)

Initial measurements were collected on 3 July 2026 on a dedicated `metrics/collection-run` branch, using host Python 3.11 with Groq-hosted inference. Infrastructure services (PostgreSQL, Redis, Qdrant, Neo4j, Prometheus) were unavailable during this run due to a local Docker daemon fault, so the pipeline executed in its in-memory LIGHT fallback mode; infrastructure-dependent metrics are listed explicitly as data gaps below.

Verified measurements:

Measurement	Result
Test suite (pytest)	360 passed, 2 skipped, 0 failed (342 s)
Statement coverage	52.4%
Component benchmarks	6/6 passed: AST chunker, token counter, token budget, Git analyzer, ContextBundle, Celery tasks
Demo pipeline (LIGHT mode)	Completed end-to-end: DebuggerAgent confidence 0.80, ReviewerAgent quality score 0.70, regression risk rated high, verdict <code>APPROVE WITH COMMENTS</code> ; 3 LLM calls (Groq)

The demo run also provides a concrete illustration of the autonomy gates functioning as designed: with confidence 0.80 (below the 0.88 auto-approve threshold) and a high risk rating, the generated fix did not qualify for autonomous merge and would route to human review — the intended conservative behavior.

Follow-up run with restored infrastructure (4 July 2026)

After Docker recovery (root cause: disabled hypervisor launch type, resolved via `bcdedit /set hypervisorlaunchtype auto` and re-enabling Virtual Machine Platform), a calibration batch of **18 diverse pipeline runs** with varied stack traces was executed against the full 10-container dev stack. Methodology:

commit `4471144`, model Llama-3.3-70b-versatile via Groq, 8 s inter-run delay to respect API rate limits. The E2E suite was also executed and surfaced one genuine codebase defect: test E5 initially failed on a `NoneType`-handling bug in `agents/debugger.py` (`commit_sha=None` passed to `str.replace`; Python's `dict.get(key, default)` covers missing keys but not present-but-`None` values). The defect was root-caused, fixed on a dedicated branch (coalescing all such patterns in the prompt builder), and verified: the full E2E suite subsequently passed **7/7**, with the 360-test unit suite unaffected. Measured from PostgreSQL (read-only queries):

Measurement	Result
Errors ingested (by severity)	32 total: 17 medium, 11 high, 4 critical (all via direct ingestion)
Fixes generated (by final status)	17 total: 15 pending human review, 2 accepted
Auto-approve rate	0.0% (0/17) — sandbox validation could not pass in this environment due to a missing <code>testcontainers</code> dependency, so no fix could satisfy all four gate conditions

Initial confidence-calibration sample (N = 18 outcomes):

DebuggerAgent confidence bucket	Count	Observed success rate	Outcomes
0.00 – 0.35	0	—	—
0.35 – 0.50	0	—	—
0.50 – 0.70	3	66.7%	2 fixed, 1 human review
0.70 – 0.88	10	100%	10 fixed
≥ 0.88	5	40.0%	2 fixed, 3 escalated

Even in this small and partially confounded sample, observed success did **not** increase monotonically with model confidence: the 0.70–0.88 bucket outperformed the ≥ 0.88 bucket. The ≥ 0.88 failures are attributable primarily to sandbox environment faults (missing `testcontainers` package, missing files) rather than reasoning errors — but this is precisely the point: self-reported confidence carries no information about *environmental* failure modes, which is why gate decisions anchored to it require empirical calibration (Section 7, failure mode 1).

Instrumentation gaps surfaced by this run (now themselves documented findings):

1. Self-healing iteration counts live in the in-memory `PipelineResult` and are persisted only when the auto-approve flow completes — failed-validation runs leave no record.
2. Security-retry counts are logged to stdout only; no audit event or database column captures them.
3. Escalation reasons are not decomposed by trigger type in the database.
4. Prometheus counters were empty because the batch ran as a standalone host process outside the FastAPI service, so the in-process registry was never scraped.



TODO (next measurement cycle): (1) install `testcontainers` so sandbox validation can actually pass — auto-approve rate and self-healing distributions are unmeasurable until then; (2) persist the four in-memory metrics above (legitimate instrumentation improvement, not test-gaming); (3) run batches through the FastAPI service (or a Prometheus push gateway) so pipeline counters are scraped; (4) grow the calibration sample toward $N \geq 50$ with sandbox faults fixed, then re-bucket.

8.2 Planned Evaluation

Track 1 — SWE-bench Lite (comparability). Adapt the ingestion layer to accept SWE-bench issue records and run the pipeline on the 300-instance Lite split with the official harness. Report resolve rate, localization accuracy (correct file in context bundle), patch application rate, and gate behavior (auto-approve vs. escalation distribution). Compare against published Agentless, SWE-agent, and AutoCodeRover numbers. If compute-constrained, run a documented random subset (≥ 50 instances).

Track 2 — Runtime-error resolution (DARA's home ground). Replay recorded production-style error traces through the full pipeline and measure end-to-end resolution rate, mean pipeline duration (`dara_pipeline_duration_seconds`), self-healing iteration counts, security-retry frequency, and escalation precision (what fraction of escalations a human judged necessary).

Ablations: (1) remove the pattern-library fast path; (2) replace Qdrant semantic retrieval with BM25; (3) disable self-healing iterations (single-shot fixing); (4) vary the auto-approve threshold (0.7–0.95) and measure the autonomy/safety trade-off curve.

9. Limitations

- No public benchmark results yet (Section 8.2 is the plan).
- Confidence thresholds (0.88 / 0.35) were set by judgment, not tuned against measured outcomes.
- The RLHF feedback loop currently *records* outcomes; strategy improvement from this data is partially implemented (memory consolidation) and fine-tuning export remains on the roadmap.
- Sandbox validation is bounded by repository test quality — an inherited, not solved, limitation.
- Instrumentation gaps: self-healing iteration counts, security-retry frequency, and escalation trigger types are held in memory and not persisted unless the auto-approve flow completes; Prometheus counters are scraped only when the pipeline runs inside the FastAPI service process (Section 8.1).
- Single-repository focus per instance; multi-repo support is roadmap.

10. Future Work

1. **Calibrated gating** — replace self-reported confidence with conformal or otherwise calibrated scores learned from logged fix outcomes, giving the auto-approve/escalate decision statistical guarantees.
2. **Rationale-aware verification** — evaluate the *reasoning* behind patches (consistency-based checks), not just test outcomes, to close the overfitted-patch gap.
3. **Topology-aware retrieval** — fuse the Neo4j service/dependency graph into context building, moving beyond flat vector similarity toward structure-aware (hypergraph) retrieval.
4. **Offline policy learning from trajectories** — every pipeline run yields a full decision trajectory with outcome labels; offline RL over this log could learn

better orchestration and strategy-selection policies than hand-set rules.

References



TODO: Full citations after reading: SWE-bench (Jimenez et al., ICLR 2024), SWE-agent (Yang et al., NeurIPS 2024), AutoCodeRover (Zhang et al., ISSTA 2024), Agentless (Xia et al., 2024), plus one RCA/AIOps survey and one calibration/conformal reference for Sections 7–10.



Remaining work, in priority order: (1) fix the local Docker daemon and re-run metrics collection to fill the §8.1 data gaps (Prometheus metrics, PostgreSQL history, confidence-vs-outcome calibration CSV); (2) run Track 1 or Track 2 evaluation, even subsampled; (3) write concrete failure-mode examples for §7; (4) read + cite the core papers; (5) export as PDF for professors. Architecture, safety design, and initial measurements (§8.1) are now grounded in the repository and real runs.